

Practical 9

Task1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Step 1 — Create the source file

nano source.txt

```
GNU nano 7.2 source.txt *
This is Week 9 OSSP practical.
We are implementing a file copy utility using Linux file I/O system calls.
This file will be copied using open, read, write, lseek and close.
We will compare it with standard library functions.
```

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OSSP
mkdir Week9
cd Week9
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ pwd
/home/jhanasri/OSSP/Week9
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ls
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ nano source.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ls -l
total 4
-rw-r--r-- 1 jhanasri jhanasri 225 Sep 21 14:32 source.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cat source.txt
This is Week 9 OSSP practical.
We are implementing a file copy utility using Linux file I/O system calls.
This file will be copied using open, read, write, lseek and close.
We will compare it with standard library functions.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ █
```

Step 2 — Create the low-level file copy program

nano lowlevel_copy.c

```
GNU nano 7.2 lowlevel_copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main() {
    int source, destination;
    char buffer[BUFFER_SIZE];
    ssize_t bytesRead, bytesWritten;

    source = open("source.txt", O_RDONLY);

    if (source == -1) {
        perror("Error opening source file");
        return 1;
    }

    destination = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination == -1) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, BUFFER_SIZE)) > 0) {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead == -1) {
        perror("Error reading source file");
    }

    /* Move file position back to the beginning */
    lseek(source, 0, SEEK_SET);

    close(source);
    close(destination);

    printf("File copied successfully using low-level system calls.\n");

    return 0;
}
```

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ nano lowlevel_copy.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ gcc lowlevel_copy.c -o lowlevel_copy
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ls
lowlevel_copy lowlevel_copy.c source.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ./lowlevel_copy
File copied successfully using low-level system calls.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ./lowlevel_copy
File copied successfully using low-level system calls.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cat destination.txt
This is Week 9 OSSP practical.
We are implementing a file copy utility using Linux file I/O system calls.
This file will be copied using open, read, write, lseek and close.
We will compare it with standard library functions.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cmp source.txt destination.txt
```

Step 3 - Standard library file copy

nano library_copy.c

```

GNU nano 7.2                                library_copy.c *
#include <stdio.h>

#define BUFFER_SIZE 1024

int main() {
    FILE *source, *destination;
    char buffer[BUFFER_SIZE];
    size_t bytesRead, bytesWritten;

    source = fopen("source.txt", "rb");

    if (source == NULL) {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen("library_destination.txt", "wb");

    if (destination == NULL) {
        perror("Error opening destination file");
        fclose(source);
        return 1;
    }

    while ((bytesRead = fread(buffer, 1, BUFFER_SIZE, source)) > 0) {
        bytesWritten = fwrite(buffer, 1, bytesRead, destination);

        if (bytesWritten != bytesRead) {
            perror("Error writing to destination file");
            fclose(source);
            fclose(destination);
            return 1;
        }
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library functions.\n");

    return 0;
}

```

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ nano library_copy.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ gcc library_copy.c -o library_copy
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ./library_copy
File copied successfully using standard library functions.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ls -l
total 52
-rw-r--r-- 1 jhanasri jhanasri  225 Sep 21 14:35 destination.txt
-rwxr-xr-x 1 jhanasri jhanasri 16232 Sep 21 14:38 library_copy
-rw-r--r-- 1 jhanasri jhanasri   959 Sep 21 14:38 library_copy.c
-rw-r--r-- 1 jhanasri jhanasri   225 Sep 21 14:38 library_destination.txt
-rwxr-xr-x 1 jhanasri jhanasri 16272 Sep 21 14:35 lowlevel_copy
-rw-r--r-- 1 jhanasri jhanasri   1182 Sep 21 14:34 lowlevel_copy.c
-rw-r--r-- 1 jhanasri jhanasri   225 Sep 21 14:32 source.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cat library_destination.txt
This is Week 9 OSSP practical.
We are implementing a file copy utility using Linux file I/O system calls.
This file will be copied using open, read, write, lseek and close.
We will compare it with standard library functions.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cmp source.txt library_destination.txt

```

Step 4 — Add performance measurement

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ dd if=/dev/zero of=large_source.txt bs=1M co
unt=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0480603 s, 2.2 GB/s
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ls -lh large_source.txt
-rw-r--r-- 1 jhanasri jhanasri 100M Sep 21 14:45 large_source.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cp large_source.txt source.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ time ./lowlevel_copy
File copied successfully using low-level system calls.

real    0m0.230s
user    0m0.034s
sys     0m0.175s
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ time ./library_copy
File copied successfully using standard library functions.

real    0m0.127s
user    0m0.016s
sys     0m0.183s
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cmp source.txt destination.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cmp source.txt library_destination.txt

```

Step 5 — Performance comparison

Method Real Time User Time System Time Low-level open/read/write

0.230 s 0.034 s 0.175 s Standard library fopen/fread/fwrite **0.127 s** 0.016

s 0.103 s

Observation: A file copy utility was implemented using low-level Linux system calls such as `open()`, `read()`, `write()`, `lseek()`, and `close()`. Another file copy utility was implemented using standard C library functions such as `fopen()`, `fread()`, `fwrite()`, and `fclose()`. A 100 MB file was used to compare their performance. The low-level implementation took 0.230 seconds, while the standard library implementation took 0.127 seconds. Both copied files were verified using the `cmp` command and were found to be identical to the source file.

Result: The file copy utility was successfully implemented using both low-level Linux file I/O system calls and standard library functions. Their performance was successfully measured and compared. In this test, the standard library implementation was faster than the low-level implementation.

Task2: Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

Step 1 — Create the Task 2 program

nano io_redirect.c

```

GNU nano 7.2                                io_redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int input_fd, output_fd;

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1) {
        perror("Error opening input file");
        return 1;
    }

    output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_fd == -1) {
        perror("Error opening output file");
        close(input_fd);
        return 1;
    }

    dup2(input_fd, STDIN_FILENO);
    dup2(output_fd, STDOUT_FILENO);

    close(input_fd);
    close(output_fd);

    printf("This output is redirected to output.txt\n");

    return 0;
}

```

Step 2 — Create the input file

nano input.txt

```

GNU nano 7.2                                input.txt *
This is Week 9 Task 2.
This file is used as the redirected standard input.
The dup2() system call is used for I/O redirection.

```

```

jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ nano ha_rename.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ gcc ha_rename.c -o ha_rename
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ./ha_rename
Week 9 OSSP Practical
File 1: file.txt
File 2: new_file.txt
The file is renamed successfully using rename() system call.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ls
ha_rename  ha_rename.c  new_file.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ mv new_file.txt ../Week8/moved_file.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ cd ../Week8
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week8$ ls
moved_file.txt
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week8$ cat moved_file.txt
This is Week 9 Task 2.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week8$ █

```

Step 3 — Modify the program to redirect both stdin and stdout
 nano io_redirect.c

```

GNU nano 7.2                                io_redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int input_fd, output_fd;
    char buffer[100];

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1) {
        perror("Error opening input file");
        return 1;
    }

    output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_fd == -1) {
        perror("Error opening output file");
        close(input_fd);
        return 1;
    }

    /* Redirect standard input to input.txt */
    dup2(input_fd, STDIN_FILENO);

    /* Redirect standard output to output.txt */
    dup2(output_fd, STDOUT_FILENO);

    close(input_fd);
    close(output_fd);

    /* Read from redirected stdin */
    while (fgets(buffer, sizeof(buffer), stdin) != NULL) {
        printf("%s", buffer);
    }

    return 0;
}

```

```
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ nano io_redirect.c
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ gcc io_redirect.c -o io_redirect
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ ./io_redirect
This is Week 9 Task 3.
This file is used as the constructed standard input.
The output file will be used for I/O redirection.
jhanasri@LAPTOP-1RN1M8E3:~/OSSP/Week9$ █
```

Observation: A C program was developed to demonstrate input and output redirection using file descriptors and the dup2() system call. The standard input file descriptor (0) was redirected to input.txt, and the standard output file descriptor (1) was redirected to output.txt. The program successfully read data from input.txt through standard input and wrote it to output.txt through standard output. This demonstrates how shells implement I/O redirection.

Result: The program successfully demonstrated standard input and output redirection using file descriptors and dup2(). The experiment showed how input can be taken from a file instead of the keyboard and output can be written to a file instead of the terminal, demonstrating the basic mechanism used by Linux shells for I/O redirection.